ESP32 Based Wireless Thermal Comfort Sensing Nodes

System Documentation

Sarvadnya Desai (210040138), Tanmay Rane (21D070080),
Shrey Agarwal (21D170039), Aryan Shah (210D170008)

November 29, 2024

Contents

1 Overview **4**
1.1 Objective . . . 4
1.2 System Architecture . . . 4

2 Detailed Mechanical Design **5**
2.1 Mechanical Design Choices . . . 5

3 Detailed Electronic Design **6**
3.1 Choice of MCU . . . 6
3.2 Choice of Sensors . . . 7
3.2.1 DS18B20 Temperature Sensor . . . 7
3.2.2 HDC1008 Temperature & Relative Humidity Sensor . . . 7
3.2.3 DHT11 RH Sensor . . . 7
3.3 Component Layout & Measurement Considerations . . . 8
3.4 Measuring Radiant Temperature . . . 8
3.5 Wireless Node Power Management . . . 8
3.6 Theoretical Power Calculation . . . 8
3.7 PCB Design . . . 9

4 Embedded Software Documentation **9**
4.1 HDC1008 Code Overview . . . 9
4.2 DS18B20 + DHT11 Code Overview . . . 12
4.3 Drivers . . . 14
4.3.1 HDC1008 Temperature & Relative Humidity Sensor . . . 14
4.3.2 DS18B20 Temperature Sensor . . . 14
4.3.3 DHT11 RH Sensor . . . 15
4.4 Abstractions Used . . . 15
4.4.1 Low-Power Deep Sleep . . . 15
4.4.2 ESP32 Wifi Communication . . . 15
4.5 Website . . . 15
4.6 Web Server Specification . . . 17
4.7 Network Design Choices . . . 17
4.8 The MQTT Protocol . . . 17

5 Practical Implementation **24**
5.1 Experimentation With Sensor Placements . . . 24
5.2 Testing and Calibration of the sensors . . . 24

6 Inference and Scope for Improvement **24**
6.1 Development in Testing Methodology . . . 24
6.2 Difficulties with MQ-135 . . . 24

7 Future Directions & BTP-2 **25**
7.1 Migrating Software Stack to ESP-IDF . . . 25
7.2 Testing . . . 25
7.3 On-spot Charging . . . 25
7.4 Devising a Control Law to Monitor Comfort . . . 26
7.5 Aspects of System Security . . . 26

8 Bill of Material **27**

9 References **29**

1 Overview

1.1 Objective

Our aim with this project is to create a **low-cost**, **power-efficient** and **durable** room monitoring node which can connect to a central server over a wireless network and publish relevant data.

This data can now be used for data analytics such as constructing a thermal/ventilation model of the room, virtual sensing at different spots in the room or even devise a control law for the ventilation and air conditioning of a room.

1.2 System Architecture

We plan on building a server-client like architecture for our wireless communication system. Our system interaction would include 3 types of devices:

1. **Central Server:** It is the device which will host the web application which allows all devices of exchange information via MQTT, provide feedback on comfort and host the front end.

2. **Wireless Sensor Nodes:** Powered by an ESP32, these nodes form the building blocks of our system. They are positioned in the room in accordance to the standards specified by thermal data collection repositories such as ASHRAE.

3. **Client Smartphones:** These devices will connect to our wireless network via a QR code and submit an interactive review of the comfort conditions in a room.

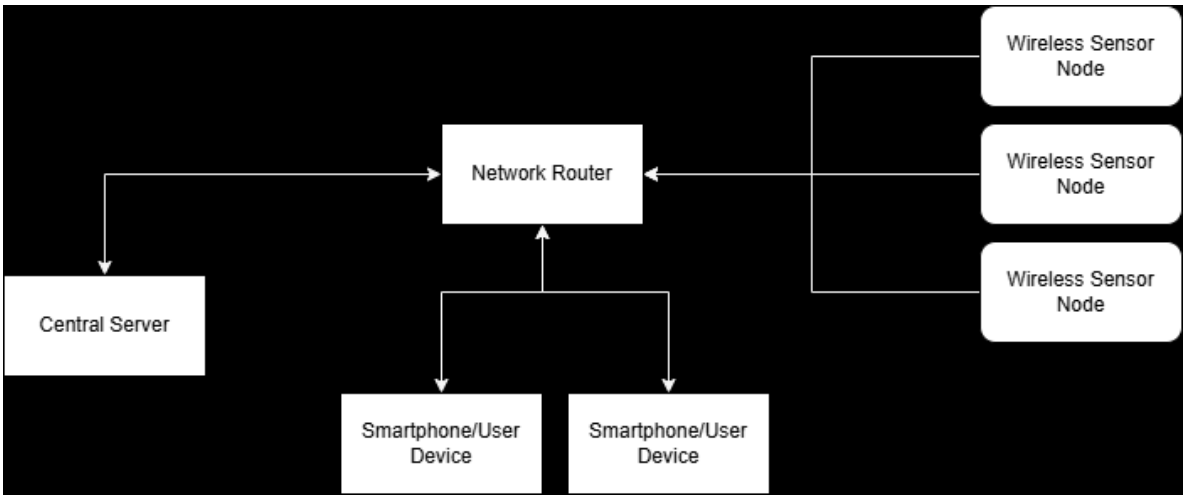

Figure 1: An overview of the Wireless Network structure

2 Detailed Mechanical Design

2.1 Mechanical Design Choices

There are a few key challenges with the mechanical design. The positioning of the Wifi antenna and ESP32 SoC must not interfere with the temperature sensing of the the room. In case the temperature sensor is placed too close to the SoC, our reading will be error prone as the metallic case of the ESP32 SoC may emit heat over duration of the experiment. To observe if the orientation and distance of the sensor from the wifi antenna can cause instrumentation issues, we tested each of the configuration using jumper cable extensions and have documented our results in the 'Practical Implementation' section.

Another key concern with the design of our node is ventilation. The enclosure must be designed in a manner such that the air flow through the sensor is not obstructed. The placement of the sensor too should be such that it doesn't remain isolated from the room inside its casing. If so, the reading of the temperature would be measured higher than expected.

The sensor should also be placed away from the battery as continuous discharge of power through the battery can cause heating.

We wish to have our PCB easily slid able through the case for battery replacement and firmware updates. Thus, we plan on having our design on Solidworks so that it can be easily 3D Printed and our network can be scaled and manufactured as per requirement.

We plan on taking up this task once the testing of our electronic prototype is complete and we have access the quality of acquired data.

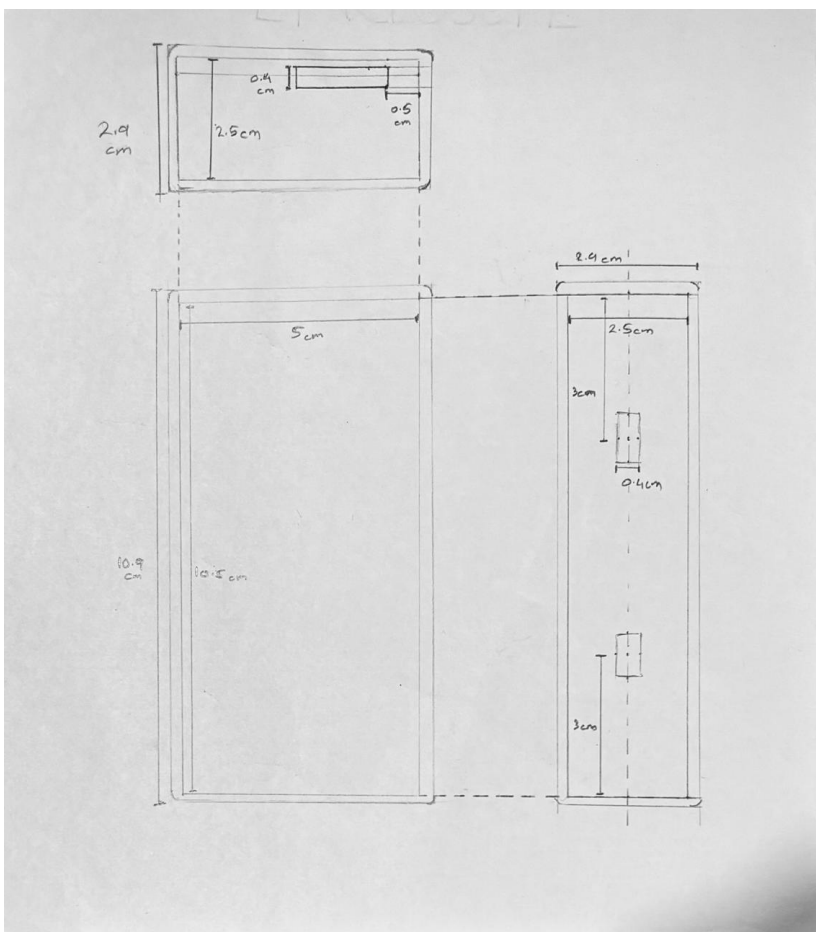

Figure 2: A Plan for the mechanical design

3 Detailed Electronic Design

3.1 Choice of MCU

We required the following capabilities for our microcontroller (MCU):

1. The MCU must have wireless capabilities to network with a central server. The Central Server can then process this data and make it available to the occupants of the room who will provide feedback to the server.

2. The MCU must be cost effective as we are to scale this network as soon as possible.

3. ADC capabilities must be present to sense and monitor the battery low threshold.

4. Peripherals such as I2C, SPI, One-Wire, etc so that the MCU can 'talk' with a wide variety of off-the-shelf sensor modules.

5. Lastly, choose an MCU which is easy to setup and configure and has a wide online support & active community.

All of these choices led us to choose the ESP32-Wroom DevKit.

1. This MCU DevKit is widely and cheaply available in the local Indian market for rapid prototyping.

2. This device has been around for more than 5 years and has extended support by Arduino IDE and various other third-party driver packages. This makes it easier to work with novel sensor modules as well.

3. This device has 2 cores which helps keep the wireless communication isolated from the sensing task and other compute processes.

4. This devices has support for a wide variety of information transmission protocols. It also has a software enable pin and a large number of ADC supported GPIO pins.

5. Due to its standardized size and on-board power management, it can be readily integrated with the external components. The small size of the MCU has also helped us bring the dimension of the sensing node to the current compact form.

6. This devices has a low power consumption deep sleep mode which helps us with the power-efficiency of our Wireless Sensor Node.

3.2 Choice of Sensors

We wish to contribute the readings obtained from our comfort monitoring sensing nodes to the ASHRAE Database. As a result, we conducted an investigation into the quality of thermal and Relative Humidity (RH) sensing performed by the contributors to this database. We have listed the Reasearch Survey papers we investigated in the 'References' section of this documentation. The following are the devices that we mainly used in this study:

- HOBO U10 Temp/RH data logger
- HOBO Pendent 64K Temperature data logger
- ExTech HT30 Heat Stress WBGT Meter
- VelociCalc Air Velocity Meter (9565 series)

The measurement of air velocity was a difficult challenge for us to integrate in a compact wireless node. We will have to thus rely on commercially available measurement meters for this as the sensitive measurement of air velocity remains a with ubiquitous air velocity measurement devices.

3.2.1 DS18B20 Temperature Sensor

The DS18B20 Temperature is an inexpensive temperature sensor which has temperature accuracy of $\pm 0.5^0$C in the atmospheric temperature range. It is available as a 3-pin package, with 1 single line of data which can be communicated with using the one-wire protocol. In case we are able to achieve the accuracy of reading practically, we can eliminate the sensor cost bottleneck in our design and roughly reduce the cost of a single node by 50%.

NOTE: The sensor needs to be powered via the 'Vin' pin on the ESP32 instead of the '3V3' pin even though the datasheet mentions that it can be powered anywhere between 3.0V to 5.5V

Link to the Data sheet

3.2.2 HDC1008 Temperature & Relative Humidity Sensor

The HDC1008 is a high resolution & high precision Temperature and Humidity sensor. It has a temperature accuracy of $\pm 0.2^0$C in the atmospheric temperature range and an relative humidity accuracy of $\pm 4\%$. This compact package is suitable for HVAC as well as smart thermostats and room monitoring solution. The drawback of this module is its cost which bottlenecks the cost per sensing node in our room monitoring solution.

The sensor communicates data to the host via the I^2C protocol and can be given up to 4 different addresses i.e. we can 4 of these sensors in parallel on a single I^2C bus. It is available as a break-out board module with analog conditioning carried out as per the data sheet instructions.

Link to Sensor purchase

Link to the Data sheet

3.2.3 DHT11 RH Sensor

The DHT11 is a generic Temperature and Relative Humidity available in the market since a long time. While its temperature accuracy is really flawed ($\pm 2^0 C$), it happens to have a decent relative humidity accuracy of $\pm 5\%$. As a result, we can use it as a relative humidity

sensor coupled with the previously mentioned DS18B20 Sensor to cut down on cost for our sensing node. DHT11 communicate with our MCU through the one-wire protocol. The data sheet linked below provides detailed information about signal conditioning required with the sensor.

Link to the Data Sheet

3.3 Component Layout & Measurement Considerations

The layout of components on the PCB we print must be such that the thermal and electromagnetic radiations of the wireless communication module must not affect the sensor position. In an attempt to reduce the module in a compact form factor we happened to place the sensor exactly above the WiFi module. The spacing between the WiFi module and sensor seems to be orders of magnitude of the wavelength of data communication we will be using. We also conducted a test by extending the temperature module through a jumper cable extension and observe no significant change in readings. Thus we can proceed with the current distance and configuration of the sensor on board.

A second concern was about the guidelines to take a temperature & relative humidity reading. How to maintain the correct air flow through the sensor node and allow enough mixing of the air such that the readings taken do not remain isolated with the surrounding room ambient data? We have designed the PCB layout in a way such as to robustly expose the sensor to the room ventilation and maintain air circulation within the node using side vents in the mechanical design. (More of that is mentioned in the Mechanical Design section)

3.4 Measuring Radiant Temperature

To measure radiant temperature we can use the 'black globe technique'. We cover the the sensor with a dark colored tape as we know that black color absorbs more ambient heat. Alternatively, we can embed our temperature sensing module in a black globe which can be painted/colored with a black absorbing paint. Now, we can expose this setup to the ambient conditions.

3.5 Wireless Node Power Management

We have chosen a 3.7V single cell Li-ion battery to power our setup. An advantage of doing so is that we can use the internal voltage step down conversion which is easily tolerant up to 5V. In order to charge the cell inside the device itself we have mounted a TP4056 based 1A Li-ion Battery charging module. Using this we can reliably charge the cell using indicator LEDs in our sensing node with adequate over charge protection.

Link to the Data Sheet

Along with this, we have tried to implement low-power sleep mode inside of our sensing node. This is an additional measure which will help us increase the single charge battery life of the device.

3.6 Theoretical Power Calculation

Below is a rough estimate of the power consumption our device will be taking up, so that we get an estimate on how long should the battery last.

Our system exists primarily in a sleep state for roughly 59 minutes in an hour. For the remaining one minute it remains in the active state. We know that any significant power consumption will take place in the sensor(s) and the the MCU. We assume the the 3.7V power

supply is being consumed at 3.3V will linear voltage regulation i.e. the power corresponding to the 3.7V and 3.3V difference is entirely dissipated as heat. (To find the minimum duration the sensor node will work reliably) Let us analyze these power consumptions separately.

Awake current consumption:

1. Active current consumption of the of ESP32 when transmitting over WiFi is $\approx$ 240 mA. Source

2. The Active measurement current consumtion of HDC1008 Temperature and Relative Humidity Sensor is $\approx$ 300 μA. (Refer Data Sheet)

Sleep Mode Current Consumption:

1. The sleep mode current consumption of ESP32 is $\approx$ 150 μA in deep sleep mode.

2. The HDC1008 consumes a maximum of $\approx$ 300 nA of current in sleep mode.

Our battery capacity is 1500 mAh. Thus power can be calculated directly by dividing the cell charge by the avg current consumption (4mA). This gives us about 375 hours of runtime which is $\approx$ 16 days.

3.7 PCB Design

A single layered PCB was designed for the purpose of this project for quick assembly of the sensor nodes and rapid scaling of the network. This was done using the KiCAD PCB design software. Relevant PCB design and schematic design files can be found in the project repository.

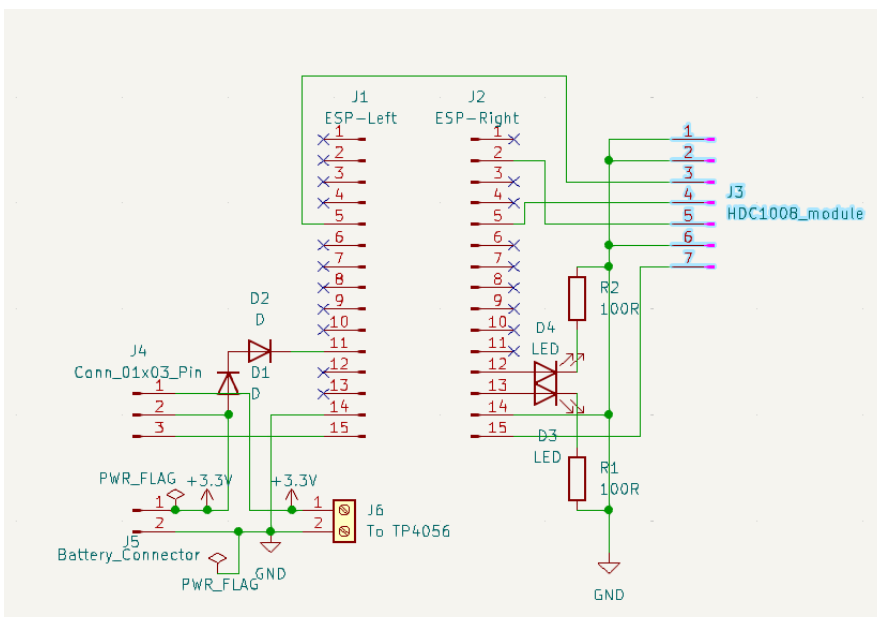

Figure 3: A view of the schematic Circuit

4 Embedded Software Documentation

4.1 HDC1008 Code Overview

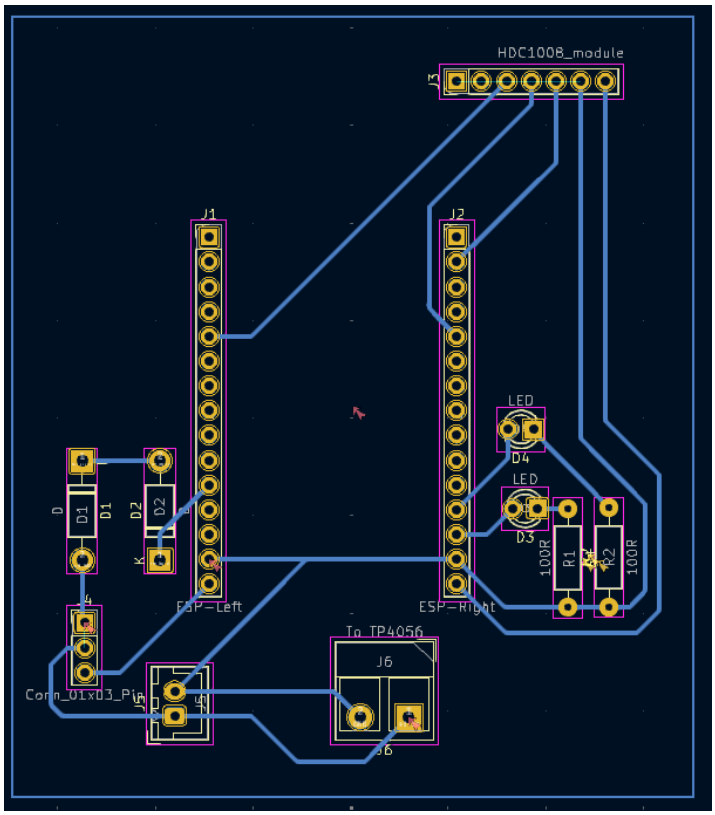

Figure 4: A view of the PCB copper layer

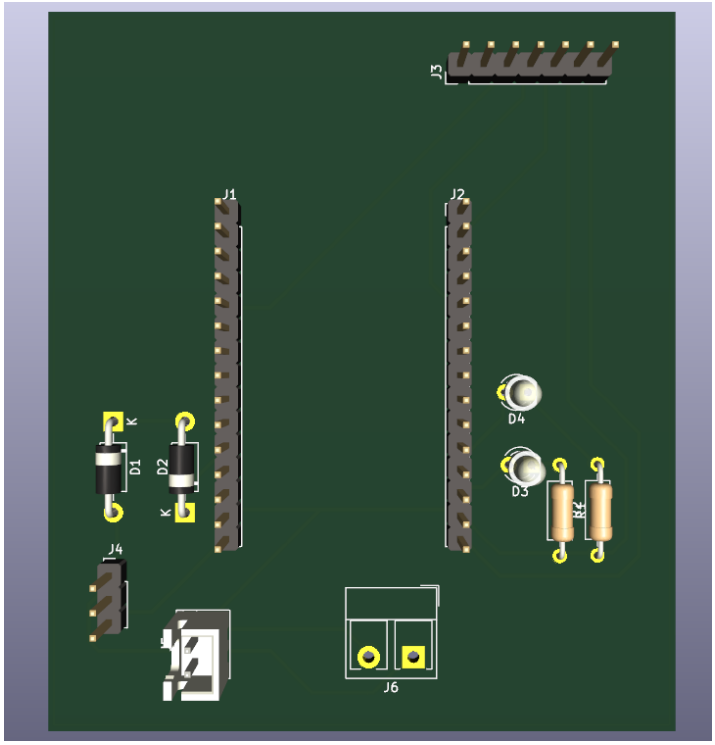

Figure 5: A 3D view of the partially populated PCB

```
    #include "EspMQTTClient.h"
#include "Adafruit_HDC1000.h"
#include <Wire.h>

#define Time_To_Sleep 30 // Time ESP32 will go to sleep (in seconds)
#define S_To_uS_Factor 1000000ULL // Conversion factor for microseconds to seconds
#define SDA_PIN 21   // Default SDA for I2C on ESP32
#define SCL_PIN 22   // Default SCL for I2C on ESP32

RTC_DATA_ATTR int bootCount = 0;
Adafruit_HDC1000 hdc = Adafruit_HDC1000();

#include <WiFi.h>

const char* ssid = "Tanmay's Galaxy M52 5G";
const char* password = "qfdq2894";

EspMQTTClient client(
  "Tanmay's Galaxy M52 5G", // WiFi SSID
  "qfdq2894",               // WiFi Password
  "broker.emqx.io",         // MQTT Broker server IP
  "emqx",                   // MQTT username (optional)
  "**********",             // MQTT password (optional)
  "TestClient",             // MQTT client name
  1883                      // MQTT port
);

void onConnectionEstablished() {
  Wire.begin(SDA_PIN, SCL_PIN);

  if (!hdc.begin()) {
    Serial.println("Couldn't find HDC100x sensor!");
    return;
  }

  float temp = hdc.readTemperature();
  float hum = hdc.readHumidity();
  String payload = String(temp) + " " + String(hum);

  if (client.publish("/sensor_data", payload.c_str())) {
    Serial.println("Published: " + payload);
  }
  else {
    Serial.println("Failed to publish data.");
  }

  WiFi.disconnect(true); // 'true' clears saved credentials
  delay(1000); // Allow some time for the disconnection

  esp_sleep_enable_timer_wakeup(Time_To_Sleep * S_To_uS_Factor);
  // Sleep after connecting and publishing
  Serial.println("Going to sleep for " + String(Time_To_Sleep) + " seconds...");
  esp_deep_sleep_start();
}

void setup() {
```

```
  Serial.begin(115200);
  delay(1000); // Allow time for Serial Monitor initialization

  /* Uncomment the below section of code to debug wifi connectivity.
    NOTE: The MQTT Library connects to WiFi by default */
  // WiFi.begin(ssid, password);
  // Serial.println("Connecting to WiFi...");

  // while (WiFi.status() != WL_CONNECTED) {
  //     delay(500);
  //     Serial.print(".");
  // }

  // Serial.println("\nWiFi connected!");
  // Serial.println("IP address: " + WiFi.localIP().toString());

  bootCount++;
  Serial.println("Boot number: " + String(bootCount));

  client.enableDebuggingMessages();
  client.enableHTTPWebUpdater();
  client.enableOTA();
  client.enableLastWillMessage("TestClient/lastwill", "I am going offline");
}

void loop() {
  client.loop();
}
```

4.2 DS18B20 + DHT11 Code Overview

```
    #include "EspMQTTClient.h"
#include <Wire.h>
#include <DS18B20.h>
#include <DallasTemperature.h>

#include "DHT.h"
#define Type DHT11
int sensepin=19;
DHT HT(sensepin,Type);
float humidity;

// Data wire is connected to GPIO23
#define ONE_WIRE_BUS 23

// Setup a oneWire instance to communicate with the DS18B20
OneWire oneWire(ONE_WIRE_BUS);

// Pass oneWire reference to DallasTemperature
DallasTemperature sensors(&oneWire);

#define Time_To_Sleep 10 // Time ESP32 will go to sleep (in seconds)
#define S_To_uS_Factor 1000000ULL // Conversion factor for microseconds to seconds

RTC_DATA_ATTR int bootCount = 0;
```

```
#include <WiFi.h>

const char* ssid = "Shrey";
const char* password = "shrey2003";

EspMQTTClient client(
  "Shrey", // WiFi SSID
  "shrey2003",              // WiFi Password
  "broker.emqx.io",        // MQTT Broker server IP
  "emqx",                  // MQTT username (optional)
  "**********",            // MQTT password (optional)
  "TestClient",            // MQTT client name
  1883                     // MQTT port
);

void onConnectionEstablished() {
  sensors.begin();
  HT.begin();

  sensors.requestTemperatures();
  temp=sensors.getTempCByIndex(0);

  hum=HT.readHumidity();

  String payload = String(temp) + " " + String((float)hum,2);

  if (client.publish("/sensor_data", payload.c_str())) {
    digitalWrite(2,HIGH);
    Serial.println("Published: " + payload);


  }
  else {
    Serial.println("Failed to publish data.");
  }

  WiFi.disconnect(true); // 'true' clears saved credentials
  delay(1000); // Allow some time for the disconnection

  digitalWrite(2,HIGH);
  esp_sleep_enable_timer_wakeup(Time_To_Sleep * S_To_uS_Factor);
  // Sleep after connecting and publishing
  Serial.println("Going to sleep for " + String(Time_To_Sleep) + " seconds...");
  esp_deep_sleep_start();
}

void setup() {
  pinMode(2,OUTPUT);
  Serial.begin(115200);
  delay(1000); // Allow time for Serial Monitor initialization

  /* Uncomment the below section of code to debug wifi connectivity.
    NOTE: The MQTT Library connects to WiFi by default */
  // WiFi.begin(ssid, password);
  // Serial.println("Connecting to WiFi...");
```

```

  // while (WiFi.status() != WL_CONNECTED) {
  //     delay(500);
  //     Serial.print(".");
  // }

  // Serial.println("\nWiFi connected!");
  // Serial.println("IP address: " + WiFi.localIP().toString());

  bootCount++;
  Serial.println("Boot number: " + String(bootCount));

  client.enableDebuggingMessages();
  client.enableHTTPWebUpdater();
  client.enableOTA();
  client.enableLastWillMessage("TestClient/lastwill", "I am going offline");
}

void loop() {
  client.loop();
}
```

4.3 Drivers

4.3.1 HDC1008 Temperature & Relative Humidity Sensor

```
#include "Adafruit_HDC1000.h"
#define SDA_PIN 21
#define SCL_PIN 22
```

#include "Adafruit_HDC1000.h": Includes the library to interface with the HDC1000 sensor.

- This library provides functions to communicate with and control the HDC1000 sensor.
- Handles I2C communication, sensor initialization, and measurement reading.
- Provides abstractions such as hdc.readTemperature() and hdc.readHumidity() to simplify access to sensor data.

#define SDA_PIN 21 and #define SCL_PIN 22: Defines the ESP32's I2C pins for data (SDA) and clock (SCL).

- **SDA (Serial Data Line):** Transmits data between the ESP32 and HDC1000.
- **SCL (Serial Clock Line):** Synchronizes communication between the master (ESP32) and slave (HDC1000).

```
Adafruit_HDC1000 hdc = Adafruit_HDC1000();
```

Creates an object HDC for the sensor to access its functions. Please refer to the module documentation linked here.

4.3.2 DS18B20 Temperature Sensor

Please refer to the module documentation linked here.

4.3.3 DHT11 RH Sensor

Please refer to the module documentation linked here.

4.4 Abstractions Used

4.4.1 Low-Power Deep Sleep

Please refer to the module documentation linked here.

4.4.2 ESP32 Wifi Communication

Please refer to the module documentation linked here.

4.5 Website

Website link : SRE-IOT Website A simple web page form is created to collect user data. ReactJS framework was used to create the frontend. Form fields are chosen based on data analysis work. The form structure is simple with four input fields:

1. **Roll number:** Text field to input the roll no. of the student. This field is required to get unique data entries about a user.
2. **Room:** This field is a dropdown menu containing all the rooms in which the modules are installed. The user will select the room they are in.
3. **Thermal sensation:** Thermal sensation values from -2 to 2 are taken through this field. Five images visualizing the thermal sensation are shown.
4. **Thermal Comfort:** Thermal comfort values are taken with values 0 (Not comfortable) and 1 (Comfortable) through thumbs up or down.

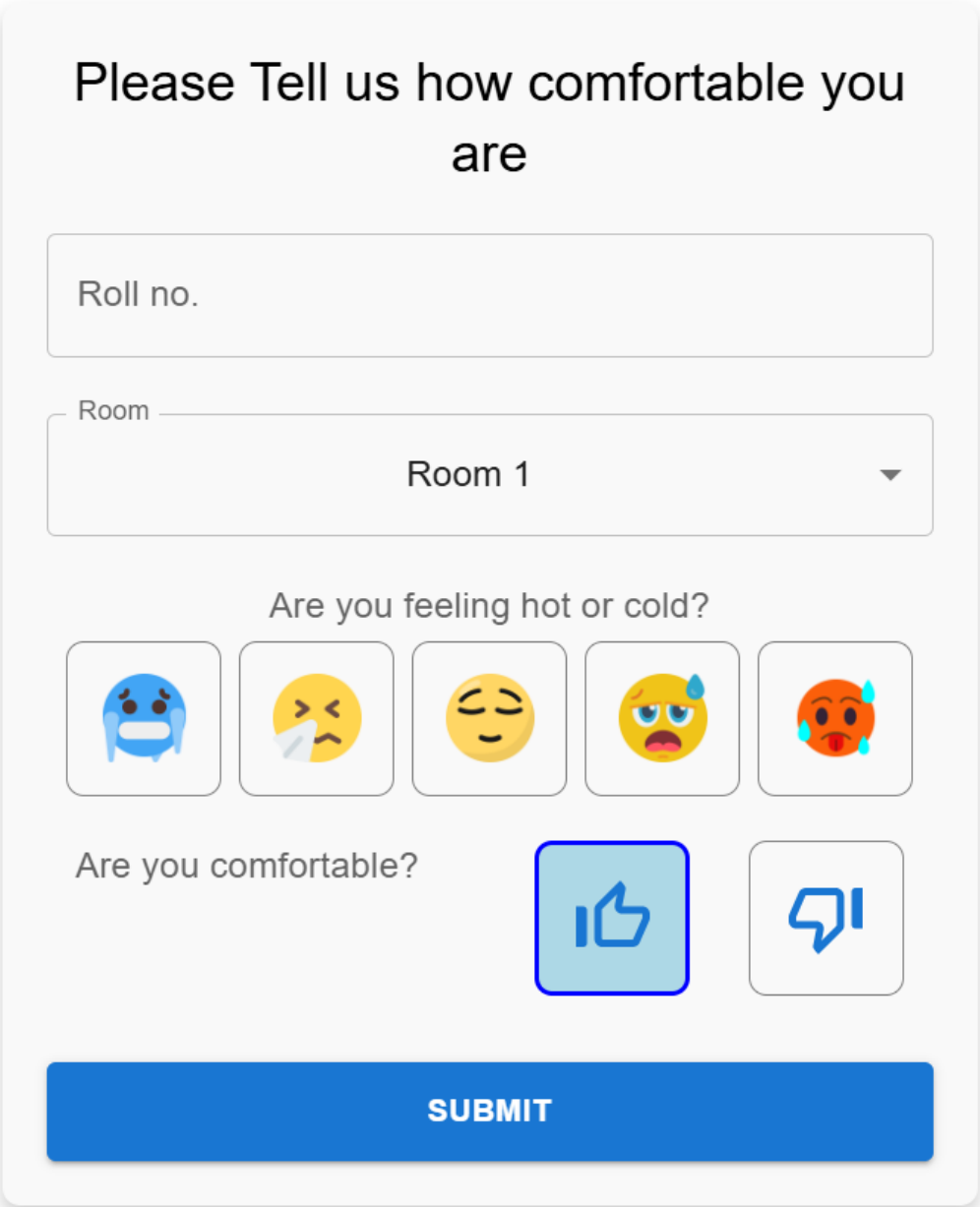

Figure 6: Webpage

4.6 Web Server Specification

A simple server was created in nodeJS to receive the data sent by the front end and, after data conversions, send it to the mqtt clients.
The data is a JSON object with the fields described above. The JSON object is converted to a string along with a timestamp, and then, using an MQTT broker, it is published to the server. A subscriber then collects the string through a Python script.

4.7 Network Design Choices

1. **Website:** For the frontend form development we used ReactJS framework as it is a easy to use frontend development framework. We used NodeJS for the backend development.

2. **Web Server:** A cloud-based web server was used to host the website. Both frontend and backend were hosted on the cloud. Vercel was used to host the frontend of the website, while Render was used to host the backend. Both are free-to-use cloud hosting services providing good enough bandwidth for our workload.

3. **Communication Protocol:** We use MQTT protocol for all the node connections, web server connections and main PC connection. We used MQTT in our application mainly because of these reasons:

   (a) **Lightweight Protocol:** MQTT is a lightweight protocol optimized for resource-constrained devices with low bandwidth, making it ideal for our application.

   (b) **Low Power Consumption:** MQTT's efficient design minimizes power usage, which is crucial for our battery-powered nodes.

   (c) **Asynchronous Communication:** MQTT enables asynchronous communication, allowing devices to send or receive data without needing a constant connection. This was crucial as if any node disconnects from the server due to any reason they should be able to connect back and again able to send messages.

   We first tried to use the Mosquitto client for our MQTT broker on the lab PC, but due to the unavailability of Wifi capabilities of the PC, we were not able to effectively use it. Also, IITB wireless made it tough to utilize the Mosquitto service. To solve this, we shifted to an open-source online MQTT broker.

4.8 The MQTT Protocol

We used Message Queuing Telemetry Transport (MQTT protocol) for node and central server connections. In this protocol rather than every node communicating with each other, they communicate with a central server called MQTT broker which receives messages from each publisher and filters them based on its topic. The messages are then sent to the subscribers, which is the main PC in our case.
The credentials for the MQTT broker connection are given in the following figure:
The web server also sends the messages to the central server using the MQTT protocol.

MQTT Broker Info

Broker Status:	Available
Broker:	broker.emqx.io
TCP Port:	1883
WebSocket Port:	8083
SSL/TLS Port:	8883
WebSocket Secure Port:	8084
QUIC Port:	14567
Certificate Authority:	broker.emqx.io-ca.crt

Figure 7: MQTT broker info.

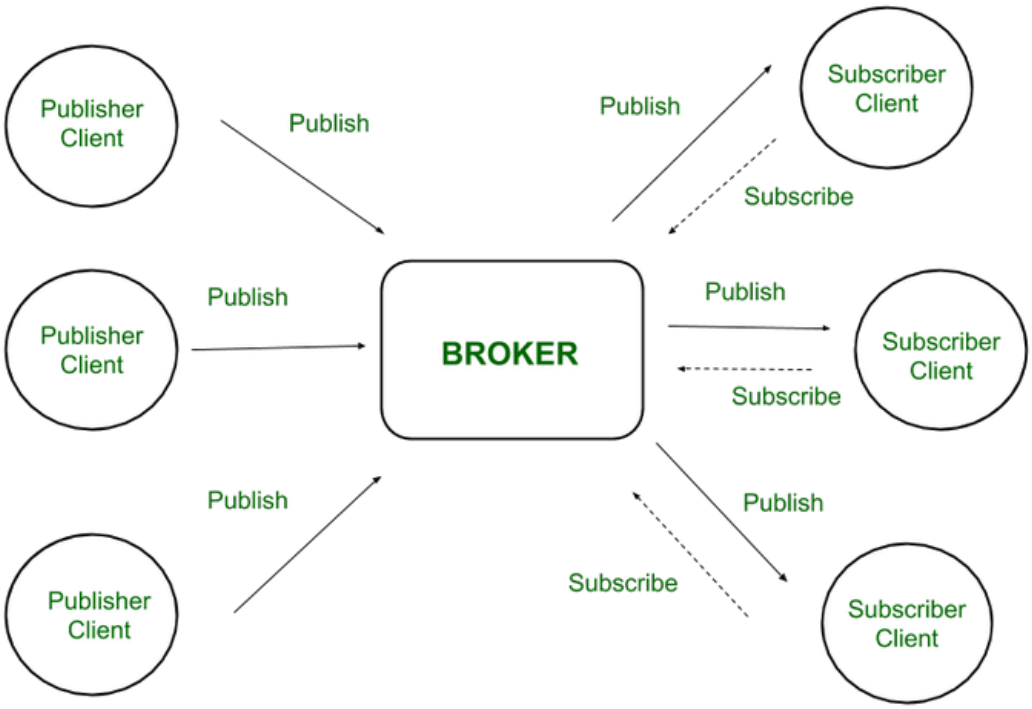

Figure 8: MQTT Protocol

Below is the script for server side connection to MQTT Broker:

```
    const express = require('express');
const mqtt = require('mqtt');
const cors = require('cors');
const os = require('os')

const app = express();
const port = process.env.PORT || 4000;

app.use(cors());
app.use(express.json());

const clientId = 'emqx_nodejs_' + Math.random().toString(16).substring(2, 8)
const username = 'emqx_test'
const password = 'emqx_test'

const client = mqtt.connect('mqtt://broker.emqx.io:1883', {
    clientId,
    username,
    password,
})

const topic = '/thermal_comfort'
const qos = 0

client.on('connect',()=>{
    console.log('Connected to MQTT');
});

app.get('/',(req,res) =>{
    console.log("Connected on port = ",port);
})

app.post('/data', (req, res) => {
    const data = req.body;

    data.timestamp = new Date().toISOString();

    console.log(data);

    // Convert the JSON object to a string before sending
    const payload = JSON.stringify(data);

    client.publish(topic, payload, { qos }, (error) => {
        if (error) {
            console.error(error);
            res.status(500).send("Error publishing message");
        } else {
            res.status(200).send("Message published successfully");
        }
    });
});
app.listen(port,()=>{
    console.log("Server running ");
})
```

Below is the script that enables the server to collect data from sensor nodes which have been connected to the MQTT Broker.

```
    import random
import json
import csv
from paho.mqtt import client as mqtt_client

broker = 'broker.emqx.io'
port = 1883
topic = "/sensor_data"
client_id = f'python-mqtt-{random.randint(0, 100)}'
username = 'emqx'
password = ''
csv_file = 'sensor_data.csv'

def connect_mqtt() -> mqtt_client:
    def on_connect(client, userdata, flags, rc):
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(client_id)
    client.username_pw_set(username, password)
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def subscribe(client: mqtt_client):
    def on_message(client, userdata, msg):
            # Parse JSON data
            # data = json.loads(msg.payload.decode())
            data = msg.payload.decode()
            print(f"Received {data} from {msg.topic} topic")

            # Write data to CSV
            write_to_csv(data)

    client.subscribe(topic)
    client.on_message = on_message

def write_to_csv(data):
    # Split the space-delimited string into individual column values
    row = data.split()

    # Check if CSV file exists and write headers if needed
    file_exists = False
    try:
        with open(csv_file, 'r'):
            file_exists = True
    except FileNotFoundError:
        pass

    # Append the processed data to the CSV file
    with open(csv_file, 'a', newline='') as f:
```

```
        writer = csv.writer(f)

        # Write the header only if the file is new
        if not file_exists:
            # Create a generic header based on the number of columns
            header = [f"Column_{i+1}" for i in range(len(row))]
            writer.writerow(header)

        # Write the row of data
        writer.writerow(row)


def run():
    client = connect_mqtt()
    subscribe(client)
    client.loop_forever()

if _name_ == '_main_':
    run()
```

Below is the script that enables the server to collect the data sent by the user over the website.

```
    import random
import json
import csv
from paho.mqtt import client as mqtt_client

broker = 'broker.emqx.io'
port = 1883
topic = "/thermal_comfort"
client_id = f'python-mqtt-{random.randint(0, 100)}'
username = 'emqx'
password = ''
csv_file = 'mqtt_data.csv'

def connect_mqtt() -> mqtt_client:
    def on_connect(client, userdata, flags, rc):
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(client_id)
    client.username_pw_set(username, password)
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def subscribe(client: mqtt_client):
    def on_message(client, userdata, msg):
        try:
            # Parse JSON data
            data = json.loads(msg.payload.decode())
            print(f"Received {data} from {msg.topic} topic")

            # Write data to CSV
            write_to_csv(data)
        except json.JSONDecodeError:
            print("Received a non-JSON message")

    client.subscribe(topic)
    client.on_message = on_message

def write_to_csv(data):
    # Check if CSV file exists and write headers if needed
    file_exists = False
    try:
        with open(csv_file, 'r'):
            file_exists = True
    except FileNotFoundError:
        pass

    # Extract fields from JSON for CSV columns
    with open(csv_file, 'a', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=data.keys())

        # Write the header only if the file is new
```

```
        if not file_exists:
            writer.writeheader()

        # Write the JSON data as a new row
        writer.writerow(data)

def run():
    client = connect_mqtt()
    subscribe(client)
    client.loop_forever()

if _name_ == '_main_':
    run()
```

5 Practical Implementation

5.1 Experimentation With Sensor Placements

We were concerned about the position of our sensor being such that if can interfere with the wireless signal sent from the ESP32 during information communication. However, we performed tests with a pair of jumper cables and positioned the sensor flexibly to see if there were any changes in the readings. We observe that there is no change in the readings when we rotate the orientation of the sensor on the perfboard prototype. There is no effect of the radio signal of the ESP32 on the reading of the sensor as well.

5.2 Testing and Calibration of the sensors

We were able to test the temperature and relative humidity sensor in a ventilated room. The readings were close to the actual temperature of the room as recorded on the AC regulator. However the least count of the AC regulator was 1^0C, thus we will try to re-record these observations in the prescence of a calibrated thermometer.

6 Inference and Scope for Improvement

6.1 Development in Testing Methodology

To tackle the task for creating fully functional sensing nodes which connect to the wireless network and log relevant sensor data, we create a step-wise testing methodology. Following from here we pursue testing the network in different settings to understand the limitations, accuracy tolerance and specifications of our device network.

Initially we tested the sensors for input reception and saw if there is a need for calibration. Based on the obtained data we observe that there is no need for direct calibration of the sensors. We then proceeded to test the data with WiFi and deep sleep enabled. We obtained positive results in latency and WiFi reconnection.

In the following time, we wish to deploy these nodes and collect the physical parameters in the setup and analyze this data by comparing it to the human feedback provided to us. We can then recalibrate the sensor network parameters to minimize the negative feedback obtained by the users.

6.2 Difficulties with MQ-135

The MQ-135 Gas sensors are used in air quality control equipments and are suitable for detecting or measuring of NH3, NOx, Alcohol, Benzene, Smoke, CO2. The sensor is capable of measuring gases in PPM an reporting analog electrical signals in a range of 0V to 5V. It is compatible with a large variety of micro controllers available in the market.

We faced issue with calibrating this sensor as we did not have an elaborate procedure to experiment with. We have yet to research ways to calibrate this sensor in order to obtain meaningful readings. Although this sensor is a great way to begin experimentation with, the accuracy of this sensor is not the best. The detection range lies in 10-300 ppm and requires ambient Relative Humidity and Temperature conditions to perform well.

Nonetheless, we plan on testing this sensor to its full capability to integrate it with the system.

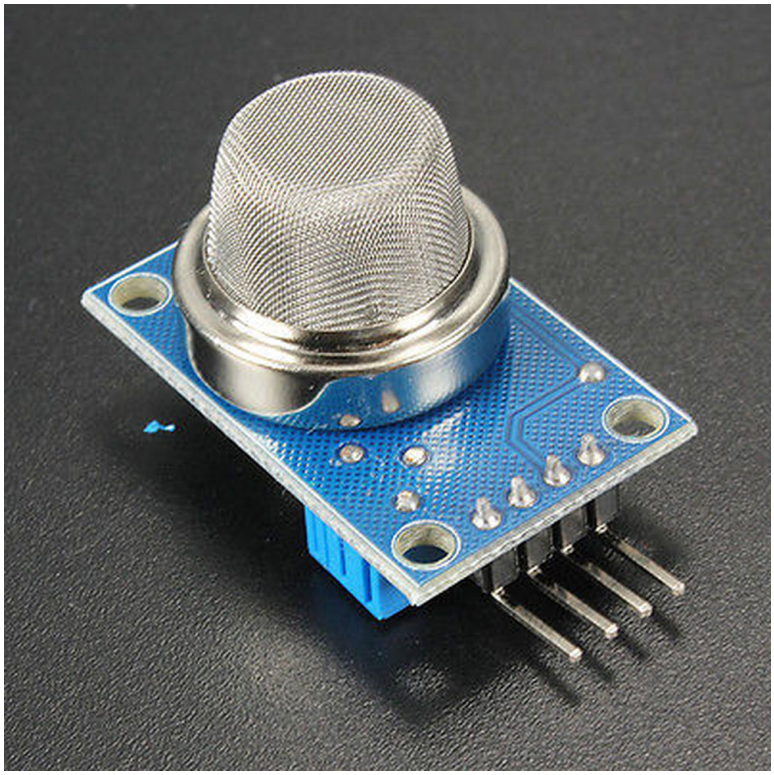

Figure 9: The MQ-135 Air Quality Sensor

7 Future Directions & BTP-2

7.1 Migrating Software Stack to ESP-IDF

Whilst we are able to access core features of the ESP32 in the Arduino plugin for ESP32, we have reason to believe we can make certain performance tweaks in the bare metal firmware to optimize our sensor power consumption and server response latency.
This is a possible avenue for us to progress in the project.

7.2 Testing

We were unable to test the entire setup for a long duration and observe performance bottlenecks. The can also test the performance of this setup across different modes of ventilation setups. Another experiment would be observe the behavior of the sensing setup across different seasons.

In either case, we were unable to extensively test the node due to time constraint.

7.3 On-spot Charging

One of the shortcomings of the setup is the downtime while charging every node. In certain environments, it would not be possible for us to find charging plug points in the room where we have setup the network. (Assume an industrial setup) Here we propose a system where we can charge a power bank and reliably supply power to the different nodes which have been discharged. This is possible as the cell capacity of the Li-ion internal battery is $\approx$ 1500 mAh and a power bank is typically 10000 mAh (even for the low end power banks).

However, there would be several challenges with this development:

- We need to have efficient power conversion from the 5V power bank to the 3.7V.
- We need to understand and probe the battery charging protocol and understand the feasibility of this solution.
- We need to understand the effect of transmission of DC power over long distances like 5 meters. (In case we have a central charge spot where we mount the battery and route power to nearby nodes)
- We need to create a provision for accurate power transfer i.e. we need to electrical connect/disconnect the power bank from the sensor node charging. We need to develop some reliable self-sustaining circuitry for this task.

7.4 Devising a Control Law to Monitor Comfort

Considering that we have an elaborate system to to sense different ventilation and thermal parameters of a room, what if we can control the room conditions based on this data. Our study shows that solely physical parameters of a room condition cannot provide valuable feedback to monitor the room temperature and ventilation. We require some user feedback as well which we have incorporated through a web application.

Once we have all this data can we use it effectively to control room ventilation elements such as Air conditioning & Heating, Fan Speed, Blind angles, Humidifiers and Air Quality monitors? We can devise an elaborate control law which can help us monitor these devices. Note that such a control law could be subjective and depends on the occupancy of the room as well.

7.5 Aspects of System Security

We have not worried about the security of our systems however the fact that our system is hosted over web, it is quite easy to manipulate the data that we publish and receive. Thus having security of our system in place is important especially after integration of feedback control and active monitoring of room ventilation devices.

We shall talk about a few basic practices that can help secure our network from intruders.

- Auto logging out of users based on geographical parameters.
- Maintaining control data at the edge effector and not transmitting it over web.
- Adding firewalls and an elaborate device log to understand if there is any threat on the network.

8 Bill of Material

The following is the bill of material for the HDC1008 based node:

Part	Price/Unit	Quantity	Price
ESP32 WROOM	425	1	425
HDC1008 Sensor Module	772	1	772
1500 mAh 3.7V LiPo Battery	449	1	449
TP4056 Charging Module	21	1	21
1N4007 Diode	59	1	59
Female Header Pins	97	1	97
SPDT Sliding Switches	49	1	49
JST-XH Connectors	65	1	65

Table 1: Bill of Material for the HDC1008 Sensor Node

The following is the bill of material for the DS18B20 based node:

Part	**Price/Unit**	**Quantity**	**Price**
ESP32 WROOM	425	1	425
DS18B20 Sensor Module	70	1	70
DHT11 Sensor Module	73	1	73
1500 mAh 3.7V LiPo Battery	449	1	449
TP4056 Charging Module	21	1	21
1N4007 Diode	59	1	59
Female Header Pins	97	1	97
SPDT Sliding Switches	49	1	49
JST-XH Connectors	65	1	65

Table 2: Bill of Material for the DS18B20 & DHT11 Sensor Node

9 References

References

[1] Honnekeri, Anoop & Brager, Gail & Dhaka, Shivraj & Mathur, Jyotirmay. (2014). Comfort and adaptation in mixed-mode buildings in a hot-dry climate.

[2] Madhavi Indraganti, Ryozo Ooka, Hom B. Rijal, Gail S. Brager, Adaptive model of thermal comfort for offices in hot and humid climates of India, Building and Environment, Volume 74, 2014

[3] Loveday, Dennis & Webb, Lynda & Verma, Purnima & Cook, Malcolm & Rawal, Rajan & Valodaria, K & Cropper, Paul & Brager, G & Zhang, Hanren & Földváry, Veronika & Arens, Edward & Babich, Francesco & Cobb, R & Ariffin, R & Kamm, S & Toledo, Linda. (2016). The role of air motion for providing thermal comfort in residential / mixed mode buildings: a multi-partner global innovation initiative (GII) project.

[4] Manoj Kumar Singh, Sadhan Mahapatra, S.K. Atreya, Thermal performance study and evaluation of comfort temperatures in vernacular buildings of North-East India, Building and Environment, Volume 45, Issue 2, 2010

[5] Sanyogita Manu, Yash Shukla, Rajan Rawal, Leena E. Thomas, Richard de Dear, Field studies of thermal comfort across multiple climate zones for the subcontinent: India Model for Adaptive Comfort (IMAC), Building and Environment, Volume 98, 2016